Git and Github Introduction

13 September 2026 15:38

① git is a software & github is a platform (service)

② git gives checkpoints for software & allows to go anywhere in code of software

---

#git Version Control System :

- It tracks the files & control the version of the Software

- Install git bash for windows.

#git :

Git is a version control system that allows you to track changes to your files and collaborate with others

It is used to manage the history of your code and to merge changes from different branches

It is a free and open-source software that is available for Windows, macOS, and Linux. Remember, GIT is a software and can be installed on your computer.

#github :

Github is a web-based hosting service for Git repositories. Github is an online platform that allows you to store and share your code with others.

It is a popular platform for developers to collaborate on projects and to share code. It is not that Github is the only provider of Git repositories, but it is one of the most popular one

---

Version control System

Version control systems are used to manage the history of your code. They allow you to track changes to your files and to collaborate with others. Version control systems are essential for software development.

Consider version control as a checkpoint in game. You can move to any time in the game and you can always go back to the previous checkpoint. This is the same concept in software development.

#History :

Before Git became mainstream, version control systems were used by developers to manage their code.

They were called SCCS (Source Code Control System). SCCS was a proprietary software that was used to manage the history of code.

It was expensive and not very user-friendly. Git was created to replace SCCS and to make version control more accessible and user-friendly. Some common version control systems are

time in the game and you can always go back to the previous checkpoint. This is the same concept in software development.

It was expensive and not very user-friendly. Git was created to replace SCCS and to make version control more accessible and user-friendly. Some commong version control systems are Subversion (SVN), CVS, and Perforce

Basic Terminologies

```
git --version
```

```
Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git
Display all 73 possibilities? (y or n)
add                         clean                       gui                 range-diff          show
am                          clone                       help                rebase              show-branch
apply                       commit                      history             reflog              sparse-checkout
archive                     config                      init                refs                stage
askpass                     credential-helper-selector  instaweb            remote              stash
askyesno                    credential-manager          lfs                 repack              status
backfill                    describe                    log                 replace             submodule
bisect                      diff                        maintenance         request-pull        survey
blame                       difftool                    merge               reset               switch
branch                      fetch                       mergetool           restore             tag
bundle                      format-patch                mv                  revert              update-git-for-windows
checkout                    fsck                        notes               rm                  whatchanged
cherry                      gc                          prune               scalar              worktree
cherry-pick                 gitk                        pull                send-email
citool                      grep                        push                shortlog
```

```
Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git status
fatal: not a git repository (or any of the parent directories): .git

Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$
```

→ not error, it shows directory is not initialized

git Config :

```
git config --global user.email "your-email@example.com"
git config --global user.name "Your Name"
```

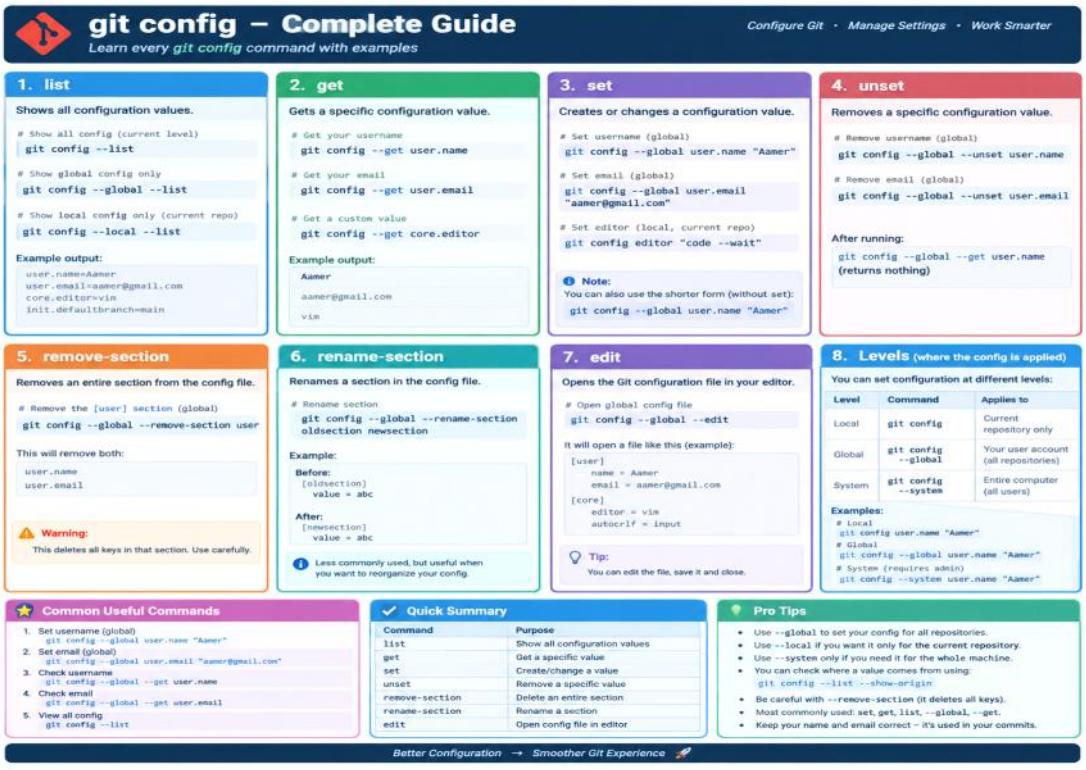
```
Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git config
edit            get             list            remove-section   rename-section   set             unset

Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git config get user.name
Aamer

Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git config get user.email
mdaamerbagban@gmail.com

Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.email=mdaamerbagban@gmail.com
user.name=Aamer

Aamer@Aamer MINGW64 ~/Desktop/Git-and-Github
$
```



* git --version
* git config → --global, --list, get

1. git init
2. .git folder
3. git status
4. git add < >
5. git commit
6. git push

Git path

13 September 2026 17:16

Repository

A repository is a collection of files and directories that are stored together. It is a way to store and manage your code.

A repository is like a folder on your computer, but it is more than just a folder. It can contain other files, folders, and even other repositories. You can think of a repository as a container that holds all your code.

There is a difference between a software on your system vs tracking a particular folder on your system. At any point you can run the following command to see the current state of your repository:

```
git status
```



Not all folders are meant to be tracked by git. Here we can see that all green folders are projects are getting tracked by git but red ones are not.

Your config settings

→ git config command

Github has a lot of settings that you can change. You can change your username, email, and other settings. Whenever you checkpoint your changes, git will add some information about your such as your username and email to the commit. There is a git config file that stores all the settings

Commit

commit is a way to save your changes to your repository. It is a way to record your changes and make them permanent.

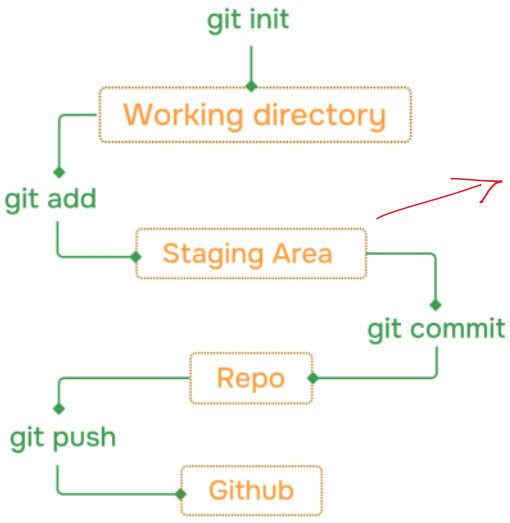
You can think of a commit as a snapshot of your code at a particular point in time. When you commit your changes, you are telling git to save them in a permanent way. This way, you can always go back to that point in time and see what you changed

Usual flow looks like this:



Complete git flow

A complete git flow, along with pushing the code to github looks like this:



When you want to track a new folder, you first use `init` command to create a new repository. Then you can use `add` command to add the folder to the repository. After that you can use `commit` command to save the changes. Finally you can use `push` command to push the changes to github. Of course there is more to it but this is the basic flow.

Stage

Stage is a way to tell git to track a particular file or folder. You can use the following command to stage a file:

```
git init
git add <file> <file2>
git status
```

Here we are initializing the repository and adding a file to the repository. Then we can see that the file is now being tracked by git. Currently our files are in staging area, this means that we have not yet committed the changes but are ready to be committed.

Commit

```
git commit -m "commit message"
git status
```

Here we are committing the changes to the repository. We can see that the changes are now committed to the repository. The `-m` flag is used to add a message to the commit. This message is a short description of the changes that were made. You can use this message to remember what the changes were. Missing the `-m` flag will result in an action that opens your default settings editor, which is usually VIM. We will change this to vscode in the next section.

Logs

```
git log
```

This command will show you the history of your repository. It will show you all the commits that were made to the repository. You can use the `--oneline` flag to show only the commit message. This will make the output more compact and easier to read.

- Check git log docs

atomic commits :

> Atomic commits are a way to make sure that each commit is a self-contained unit of work. This means that if one commit fails, you can always go back to a previous commit and fix the issue. This is important for maintaining a clean and organized history in your repository.

change default code editor

→ git's default code editor is Vim but we can use VS code

You can change the default code editor in your system to vscode. To do this, you can use the following command:

```
git config --global core.editor "code --wait"
```

gitignore

Gitignore is a file that tells git which files and folders to ignore. It is a way to prevent git from tracking certain files or folders. You can create a gitignore file and add list of files and folders to ignore by using the following command:

Example:

```
// .gitignore
```

.gitkeep :

- use to track empty folder

but → also

Example:

```
// .gitignore
node_modules
.env
.vscode
```

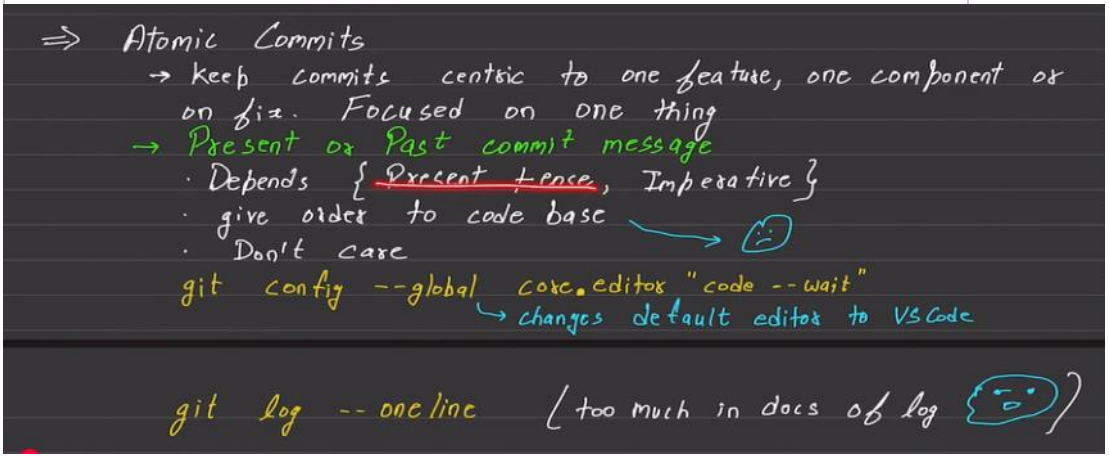
Now, when you run the `git status` command, it will not show the `node_modules` and `.vscode` folders as being tracked by git.

but also

- git does not track empty folders

Conclusion

In this section, we have learned about the basics of git and how to use it to track changes to your files and folders. We have also learned about the different commands that you can use to interact with your repository, such as `init`, `add`, `commit`, `log`, etc By the end of this section, you



(1) if we delete .git so our local repo again required to initialize git repo

(2) to unstage the file use:
git rm --cached <file>

(3) git add { file1 file2 / *

(4) to change default vim GUI use command

git config --global core.editor "code --wait"

(5) if git commit only → then → vim open for enter the commiting message

(6) git log --oneline → To check commits

message

⑥ git log --oneline → To check commits

gitignore – Specifies intentionally untracked files to ignore

A `.gitignore` file tells Git which files or directories to ignore (keep untracked).

Location (in order of precedence):
1. Command line patterns
2. `.gitignore` (in current/parent dirs)
3. `$GIT_COMMON_DIR/info/exclude`
4. `core.excludesFile`

1. What is gitignore?

A gitignore file specifies intentionally untracked files that Git should ignore. Files already tracked by Git are not affected.

Common locations

```
$XDG_CONFIG_HOME/git/ignore
$GIT_COMMON_DIR/info/exclude
.gitignore
```

Example: `.gitignore` in project root

```
# Ignore build output
/dist/
*.log
```

2. Pattern Format

Format	Meaning
(blank line)	Matches no files (separator).
`# comment`	Comment line.
\ (backslash)	Escapes special characters.
`!pattern`	Negates the pattern (re-include).
`/`	Directory separator.
`*`	Matches any string except /.
`?`	Matches any single character except /.
`[a-z]`	Matches one character in range.
`**`	Special meaning (see below).

Note: Trailing spaces are ignored unless escaped with a backslash (\).

3. Sources & Precedence

Git checks patterns in this order (highest to lowest):

1. Command line patterns (for supported commands)
2. `.gitignore` in current/parent directories
3. `$GIT_COMMON_DIR/info/exclude`
4. `core.excludesFile` (from ~/.gitconfig)

Patterns from higher levels override lower ones (within the same level, the last matching pattern wins).

Where to put patterns?

- Shared (all developers) → `.gitignore` (in repository)
- Repo specific (not shared) → `$GIT_COMMON_DIR/info/exclude`
- Personal (all situations) → `core.excludesFile` (~/.gitconfig)

4. Examples of Common Patterns

Pattern	What it matches
`*.log`	Any file ending with `.log`
`/build/`	build directory (root level only)
`temp/`	temp directory (matches anywhere)
`doc/*.txt`	`.txt` files inside doc (not subdirs)
`**/foo`	foo anywhere in the tree
`a/**/b`	a/b, a/x/b, a/x/y/b ...
`!important.txt`	Re-include important.txt
`\!file.txt`	A file literally named `!file.txt`
`foo/*`	Files/dirs directly inside foo (not foo/bar/*)

5. Special Double Star (**)

Pattern	Meaning
`**/foo`	Match foo anywhere.
`foo/**`	Everything inside foo (infinite depth).
`a/**/b`	Match b in a, a/x/b, a/x/y/b ...
`**`	Behaves like * (regular asterisk).

Example:

```
repo/
├─ a/
│  ├─ b/
│  │  └─ file.txt   ← matches a/**/b
├─ foo/
│  └─ bar.txt       ← matches **/foo
```

6. Real-World Examples

1. Ignore object and archive files (anywhere)

```
# .git/info/exclude
*.o
*.a
```

2. Ignore HTML in Documentation except foo.html

```
# Documentation/.gitignore
*.html
!foo.html
```

```
# After running git status
Documentation/foo.html
(not ignored)
```

3. Ignore everything except a specific directory

```
/*
!/foo
/foo/*
!/foo/bar
```

Keeps only foo/bar in the repository.

7. Important Notes

- To stop tracking a file that is already trackedc!
  `git rm --cached <file>`
  Then add it to .gitignore.
- **Git does not follow** symbolic links when reading .gitignore from the working tree.
- Use `git check-ignore -v <file>` to see which pattern matched.

8. Configuration Variable

`core.excludesFile` points to a file with patterns (used in addition to `$GIT_COMMON_DIR/info/exclude`).

Default location:

```
$XDG_CONFIG_HOME/git/ignore
(if not set or empty → $HOME/.config/git/ignore)
```

9. Quick Commands

Task	Command
See all config	`git config --list`
Get a value	`git config --get key`
Set a value	`git config --set key value`
Unset a value	`git config --unset key`
Edit config	`git config --edit`

gitignore → Keep your repository clean ✓

① git init

② .git folder

③ git status

④ git add

⑤ git commit

✓ -m -mx atomic change

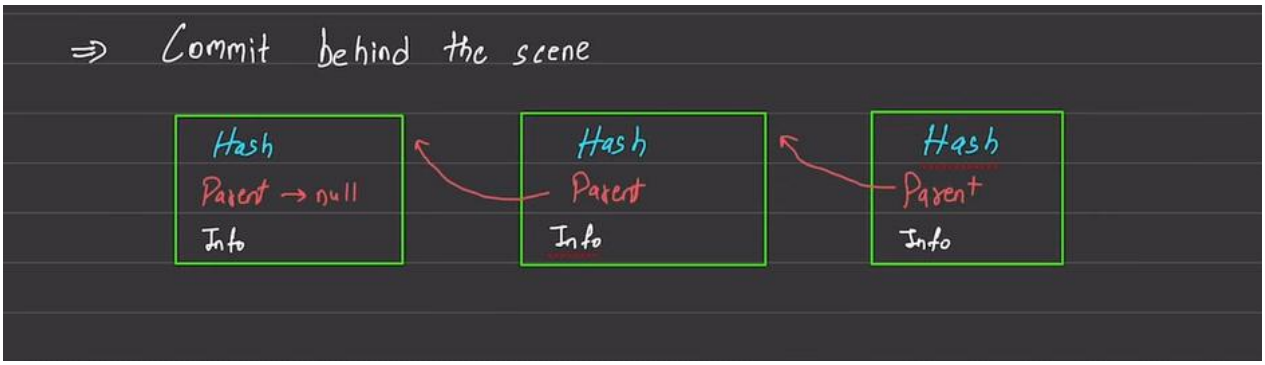
⑥ .gitignore

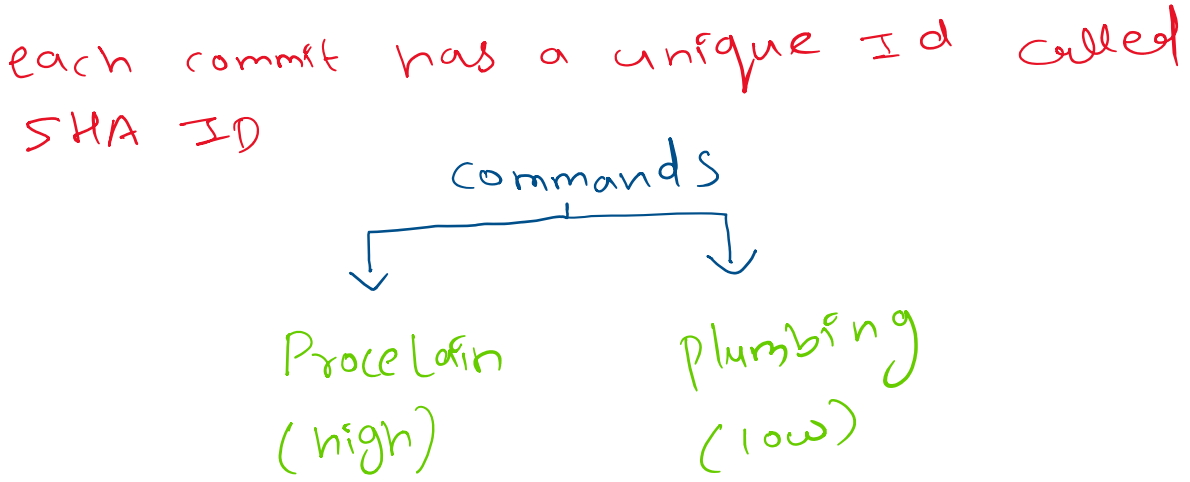
⑦ .gitkeep

⑧ git log

Behind The Scene Of Git

13 September 2026 18:46





Git is a version control system that allows you to track changes to your files and folders. It is a powerful tool that can help you manage your code more effectively. In this section, we will explore the basics of how git works internally.

Git Snapshots

A git snapshot is a point in time in the history of your code. It represents a specific version of your code, including all the files and folders that were present at that time.

Each snapshot is identified by a unique hash code, which is a string of characters that represents the contents of the snapshot.

A snapshot is not an image, it's just a representation of the code at a specific point in time.

Snapshot is a loose term that is used when git stores information about the code in a locally stored key-value based database.

Everything is stored as an object and each object is identified by a unique hash code

3 Musketeers of Git

The three musketeers of git are:

- Commit Object
- Tree Object
- Blob Object

Commit Object

Each commit in the project is stored in .git folder in the form of a commit object.
A commit object contains the following information:

- Tree Object
- Parent Commit Object
- Author
- Committer
- Commit Message

Tree Object

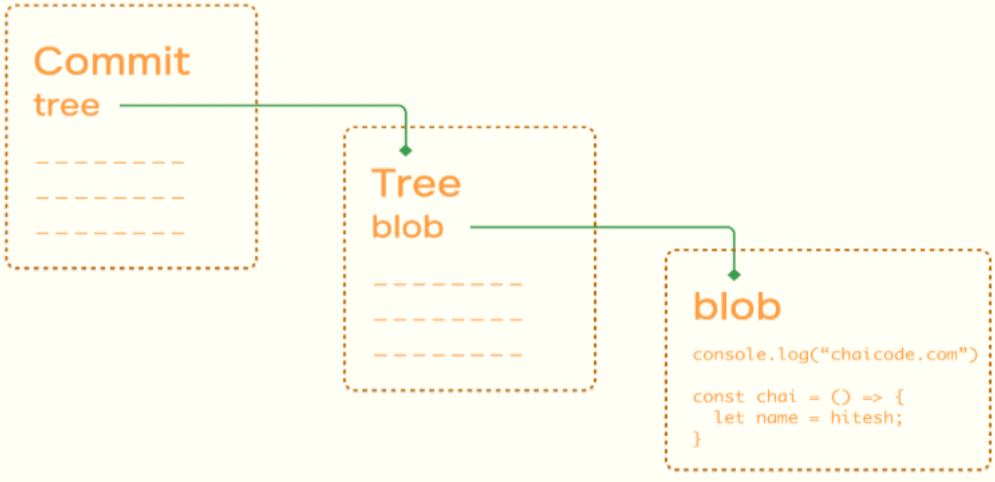
Tree Object is a container for all the files and folders in the project. It contains the following information:

- File Mode
- File Name
- File Hash
- Parent Tree Object

Everything is stored as key-value pairs in the tree object. The key is the file name and the value is the file hash.

Blob Object

Blob Object is present in the tree object and contains the actual file content. This is the place where the file content is stored.



Helpful commands

Here are some helpful commands that you can use to explore the git internals:

```
git show -s --pretty=raw <commit-hash>
```

Grab tree id from the above command and use it in the following command to get the tree object:

```
git ls-tree <tree-id>
```

Grab tree id from the above command and use it in the following command to get the blob object:

```
git show <blob-id>
```

Grab tree id from the above command and use it in the following command to get the commit object:

```
git cat-file -p <commit-id>
```

#practical of low level Commands

Understanding Git's Internal Commands

A Deep Dive into Objects, Plumbing, Porcelain, and Commit Storage Mechanics

1. How Git Stores Your Project

At its core, Git operates as a content-addressable filesystem. The easiest way to understand Git's object-level commands is first to understand how Git stores a commit visually and logically.

Commit Object

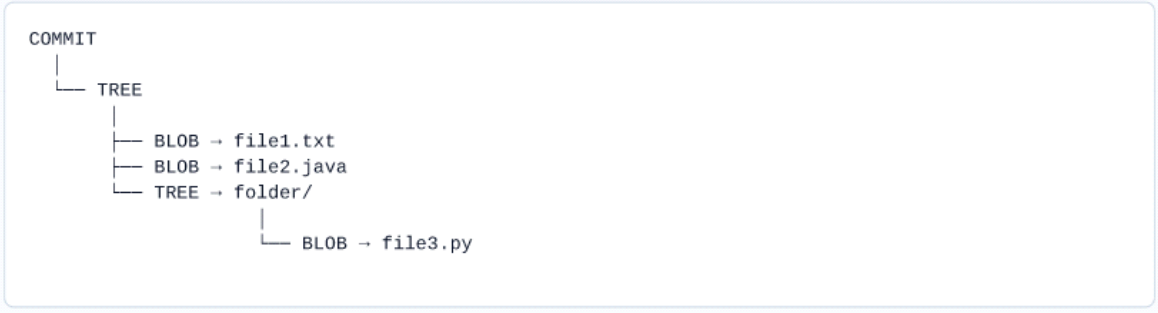
Metadata about a commit (author, timestamp, log message) + pointer to the root tree object and parent commit hash(es).

Tree Object

Represents directory/folder structures. Contains file names, permissions, and pointers to corresponding blob or nested tree IDs.

Blob Object

Stores raw file content (e.g., source code). Blobs do *not* store filenames, timestamps, or directory paths.



Relationship Hierarchy: Commit → Tree → Blob

2. Inspecting Commits: `git show -s --pretty=raw <commit-hash>`

Every object in Git is identified by a unique SHA-1 (or SHA-256) hash (e.g., `a1b2c3d4e5f6...`).

Command:

```
git show -s --pretty=raw a1b2c3d
```

Command Breakdown:

- `git show` : Normally displays commit details along with the diff/patch.
- `-s` : Suppresses the diff output so you only see the raw metadata.
- `--pretty=raw` : Forces Git to display the internal raw header layout of the commit object.

Example Output:

```
commit a1b2c3d4e5f67890123456789012345678901234
tree 8f7e6d5c4b3a2918273645546372819001928374
parent 1234567890abcdef1234567890abcdef12345678
author Aamer <aamer@example.com> 1700000000 +0000
committer Aamer <aamer@example.com> 1700000000 +0000

    Added login feature
```

The crucial piece retrieved here is the **tree ID** (`8f7e6d5c...`), which points to the root directory snapshot of this commit.

3. Inspecting Directories: `git ls-tree <tree-id>`

Using the tree ID obtained from the raw commit display, you can inspect the contents of that directory snapshot.

```
git ls-tree 8f7e6d5c
```

Example Output:

```
100644 blob 111aaa...    README.md
100644 blob 222bbb...    app.py
040000 tree 333ccc...    src
```

Understanding the entries:

- `100644 blob 111aaa... README.md` : Standard file mode (`100644`) pointing to a **blob** with hash `111aaa...` mapped to name `README.md` .
- `040000 tree 333ccc... src` : Directory mode (`040000`) pointing to a nested **tree** object `333ccc...` for directory `src/` .

4. Inspecting Files: `git show <blob-id>`

Once you have a blob ID from a tree listing, you can inspect its actual contents directly:

```
git show 111aaa
```

Output:

```
Hello Git
This is my README file.
```

> **Core Git Concept: Blobs do not know their filenames**
>
> A blob object only stores the file data contents. The filename (`README.md`) and file permissions are stored entirely within the parent **Tree** object.

5. The Universal Inspector: `git cat-file -p <object-id>`

The `git cat-file` command is Git's primary low-level inspection utility. The `-p` flag tells Git to **pretty-print** the contents based on the object's type.

```
git cat-file -p <object-id>
```

- For a **Commit ID**: Displays raw commit headers (tree, parent, author, log message).
- For a **Tree ID**: Displays directory listings (modes, types, hashes, filenames).
- For a **Blob ID**: Outputs raw contents of the file.

6. Clarification & Navigation Sequence

To navigate from a commit to raw file contents, follow this precise object lookup chain:

```
COMMIT ID (abc123)
    |
    |  git show -s --pretty=raw abc123
    ↓
 TREE ID (def456)
    |
    |  git ls-tree def456
    ↓
 BLOB ID (aaa111)
    |
    |  git show aaa111  OR  git cat-file -p aaa111
    ↓
RAW CONTENT ("Hello Git")
```

7. Complete Execution Flow Example

Suppose we have commit hash `abc123` :

1. **Inspect Commit:** `git show -s --pretty=raw abc123`

```
commit abc123
tree def456
parent 999888
author Aamer

    Initial commit
```

2. **Inspect Tree:** `git ls-tree def456`

```
100644 blob aaa111 README.md
100644 blob bbb222 main.py
```

3. **Inspect Blob:** `git show aaa111`

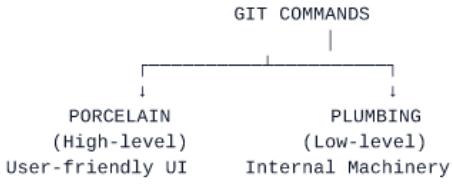
```
# My Project
This is my project.
```

Visual Mapping:

```
abc123 (commit)
   │
   └── def456 (tree)
         ├── aaa111 (blob) ──► README.md
         └── bbb222 (blob) ──► main.py
```

8. Git Architecture: Porcelain vs. Plumbing

Git commands are divided into two distinct conceptual layers based on target audience and abstractions:



Porcelain Commands

High-level commands intended for daily user workflows. They abstract away object management, hashes, trees, and index files into intuitive concepts like "saving", "updating", or "syncing".

Examples: `git status`, `git add`, `git commit`, `git push`, `git pull`, `git switch`, `git merge`, `git log`.

Plumbing Commands

Low-level utility commands designed for underlying script engines, tooling, and direct manipulation of Git's object database.

Examples: `git cat-file`, `git hash-object`, `git ls-tree`, `git update-index`, `git write-tree`, `git commit-tree`.

9. Side-by-Side Comparison

Feature	Porcelain Commands	Plumbing Commands
Abstraction Level	High-level interface	Low-level primitives
Primary Purpose	Normal daily developer workflow	Git internal machinery & scripting
User Friendliness	High (human-readable outputs)	Low (raw data, exact hashes)
Usage Frequency	Daily usage	Internal execution / Debugging
Commit Creation	`git commit`	`git commit-tree`
Status Checking	`git status`	`git cat-file` / `git ls-files`
Staging Files	`git add`	`git update-index`
History Viewing	`git log`	`git rev-list`

> **Easy Memory Analogy**
>
> **Porcelain** = What users touch (fixtures, handles, surface interface).
> **Plumbing** = What connects Git internally (pipes, valves, mechanics underneath).

10. Key Plumbing Commands Reference

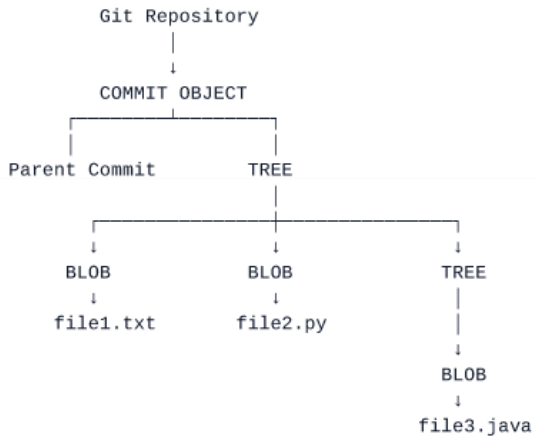
- `git cat-file -p <object-id>` — Pretty-print content of commit, tree, or blob objects.
- `git ls-tree <tree-id>` — List names, modes, and object hashes within a tree.
- `git hash-object <file>` — Calculate content SHA hash and optionally create a blob.
- `git write-tree` — Write current index/staging area into a tree object.
- `git commit-tree <tree-id> -p <parent-id>` — Create commit object directly from tree.
- `git update-index --add <file>` — Register file changes directly into the index cache.

11. Diagnostic & Tooling Commands

Commands useful for configuring, inspecting environment metadata, or repairing database integrity:

- `git --version` — Check current installed Git version.
- `git config --list` — Display operational configuration settings.
- `git check-ignore -v <file>` — Debug which rule in `.gitignore` is ignoring a file.
- `git reflog` — View log of reference pointer movements (vital for recovering lost commits).
- `git fsck` — Perform verification and connectivity checks on the object database.

12. Master Architecture Summary



Key Takeaway

Porcelain commands are for using Git; plumbing commands are for understanding and controlling Git's internal machinery.

Git Branches And Conflicts

13 September 2026 21:13

Head / Header :

- Head is a pointer pointing on current branch.
- Head is took from old tape recorder 
- Code is write where head is present
- .git file consist the HEAD file & ref folder also contains Heeed folder which shows how many branches are their

  ├ HEAD ← point to branch
  └ ref ||— HEAD ├ b1 } branches
  ||— b2

git Graph : It is a tool used for branch visulization

Branches :

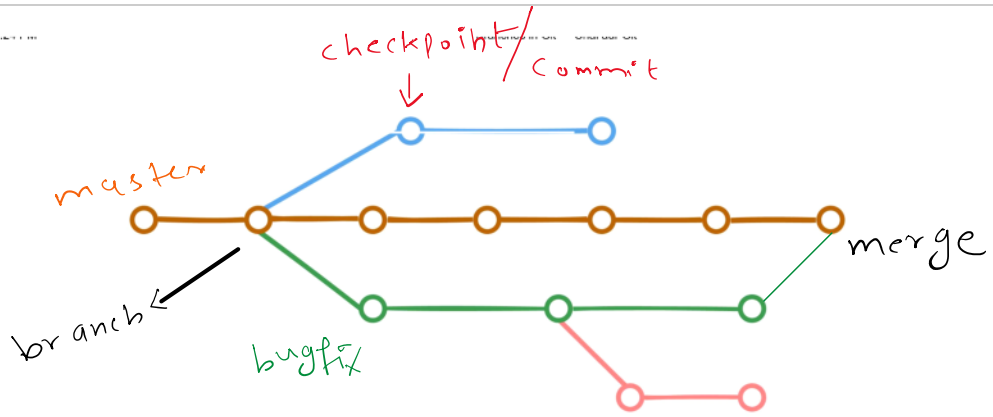
Branches are a way to work on different versions of a project at the same time.

They allow you to create a separate line of development that can be worked on independently of the main branch.

This can be useful when you want to make changes to a project without affecting the main branch or when you want to work on a new feature or bug fix.

to create branch

1. git branch <name>
2. git branch → to check which branch header is pointing
3. git checkout <name>



Some developers can work on Header, some can work on Footer, some can work on Content, and some can work on Layout. This is a good example of how branches can be used in git.

HEAD in git

The HEAD is a pointer to the current branch that you are working on. It points to the latest commit in the current branch. When you create a new branch, it is automatically set as the HEAD of that branch.

> the default branch used to be master, but it is now called main. There is nothing special about main, it is just a convention.

Creating a new branch

To create a new branch, you can use the following command:

```
git branch
git branch bug-fix
git switch bug-fix
git log
git switch main
git switch -c dark-mode
git checkout orange-mode
```

git switch <name> → for changing branch

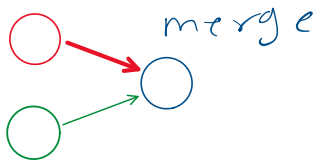
git switch -c <name> → create new branch & change branch

Some points to note:

- `git branch` - This command lists all the branches in the current repository.
- `git branch bug-fix` - This command creates a new branch called `bug-fix`.
- `git switch bug-fix` - This command switches to the `bug-fix` branch.
- `git log` - This command shows the commit history for the current branch.
- `git switch main` - This command switches to the `main` branch.
- `git switch -c dark-mode` - This command creates a new branch called `dark-mode`. the `-c` flag is used to create a new branch.
- `git checkout orange-mode` - This command switches to the `orange-mode` branch.

> Commit before switching to a branch Go to .git folder and checkout to the HEAD file

Merging branches



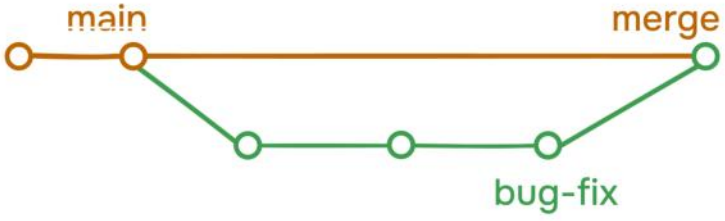
- Merging is about bringing changes from one branch to another.
- In Git we have two types of merges :
  - Fast-Forward Merges (If branches have not diverged)
  - 3-Way Merges (if branches have diverged)

Fast-forward merge

This one is easy as branch that you are trying to merge is usually ahead and there are no conflicts.

When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

```
git checkout main
git merge bug-fix
```



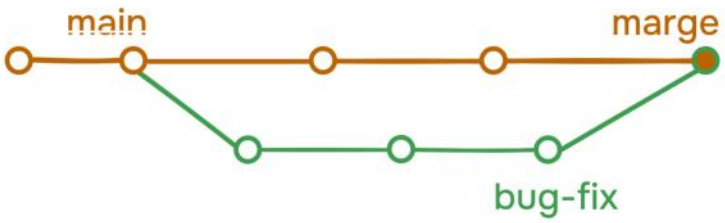
Some points to note:

- `git checkout main` - This command switches to the `main` branch.
- `git merge bug-fix` - This command merges the `bug-fix` branch into the `main` branch.

This is a fast-forward merge. It means that the commits in the `bug-fix` branch are directly merged into the `main` branch. This can be useful when you want to merge a branch that has already been pushed to the remote repository.

3 Way merge

chaidocs Library Search chaicode Search docs



In this type of merge, the main branch has additional commits that are not present in the `bug-fix` branch. This is not a fast-forward merge. Here git looks at 3 different commits [common ancestor of branches + tips of each branch] and combines the changes into one merge commit.

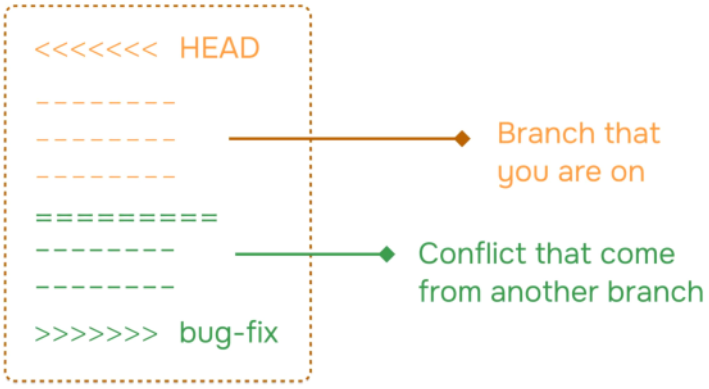
When you are done working on a branch, you can merge it back into the main branch. This is done using the following command:

```
git checkout main
git merge bug-fix
```

If the command are same, what is the difference between fast-forward and not fast-forward merge?

The difference is resolving the conflicts. In a fast-forward merge, there are no conflicts. But in a not fast-forward merge, there are conflicts, and there are no shortcuts to resolve them. You have to manually resolve the conflicts. Decide, what to keep and what to discard. VSCode has a built-in merge tool that can help you resolve the conflicts.

☆important



Managing conflicts

There is no magic button to resolve conflicts. You have to manually resolve the conflicts. Decide, what to keep and what to discard.

VSCode has a built-in merge tool that can help you resolve the conflicts. I personally use VSCode merge tool. Github also has a merge tool that can help you resolve the conflicts but most of the time I handle them in VSCode and it gives me all the options to resolve the conflicts.

Overall it sounds scary to beginners but it is not, it's all about communication and understanding the code situation with your team members.

Rename a branch

You can rename a branch using the following command:

```
git branch -m <old-branch-name> <new-branch-name>
```

Delete a branch

You can delete a branch using the following command:

```
git branch -d <branch-name>
```

Checkout a branch

You can checkout a branch using the following command:

```
git checkout <branch-name>
```

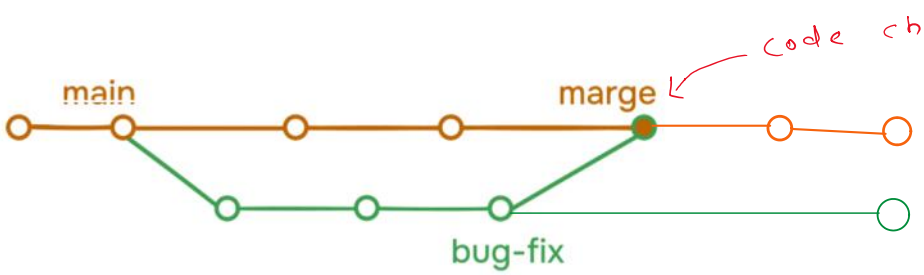
Checkout a branch means that you are going to work on that branch. You can checkout any branch you want.

List all branches

You can list all branches using the following command: git branch

☆ Important : git merge <name>

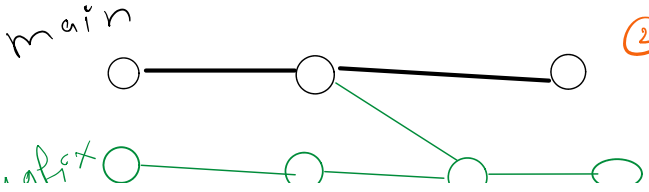
① if Head is on master then code will change in master branch only and other branch remains their code as it is



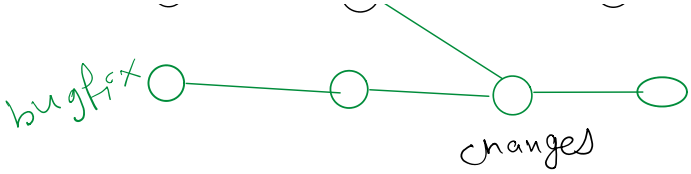
① Head → main
git merge bugfix

② if Head → on → bugfix branch then Code change in bugfix

① Head → bugfix



② git merge main


bugfix
changes

git stash , diff and tag

13 September 2026 23:03

Chai aur Git

Diff, Stash and Tags

Hitesh Choudhary 4 pages 5 min read Updated Aug 9, 2026

On this page

This guide will help you understand the different commands related to diff, tags and stash in git. These are not main stream commands but they are very useful in certain situations.

Git diff

The `git diff` is an informative command that shows the differences between two commits. It is used to compare the changes made in one commit with the changes made in another commit. Git consider the changed versions of same file as two different files. Then it gives names to these two files and shows the differences between them.

How to Read the Diff Output

- `a/` – the original file (before changes)
- `b/` – the updated file (after changes)

- `---` – marks the beginning of the original file
- `+++` – marks the beginning of the updated file
- `@@` – shows the line numbers and position of changes

Here the file A and file B are the same file but different versions.

Git will show you the changes made in the file A and file B. It will also show you the line number where the change occurred along with little preview of the change.

Comparing Working Directory and Staging Area

```
git diff
```

This command shows the unstaged changes in your working directory compared to the staging area. This command alone will not show you the changes made in the file A and file B, you need to provide options to show the changes.

Comparing Staging Area with Repository

```
git diff --staged
```

This command shows the changes between your last commit and the staging area (i.e., changes that are staged and ready to be committed).

Comparing Two Branches

```
git diff <branch-name-one> <branch-name-two>
```

This command compares the difference between two branches.

Another way to compare the difference between two branches is to use the following command:

```
git diff branch-name-one..branch-name-two
```

Comparing Specific Commits:

```
git diff <commit-hash-one> <commit-hash-two>
```

This command compares the difference between two commits.

Git Stash



Stash is a way to save your changes in a temporary location. It's useful when switching branches without losing work. You can then come back to the file later and apply the changes.

> Conflicting changes will not allow you to switch branches without committing the changes. Another alternative is to use the `git stash` command to save your changes in a temporary location.

```
git stash
```

This command saves your changes in a temporary location. It is like a stack of changes that you can access later.

Naming the stash

You can also name the stash by using the following command:

```
git stash save "work in progress on X feature"
```

View the stash list

You can view the list of stashes by using the following command:

```
git stash list
```

Apply the Most Recent Stash

You can apply the stash by using the following command:

```
git stash apply
```

Apply Specific Stash

You can apply the specific stash by using the following command:

```
git stash apply stash@{0}
```

Here `stash@{0}` is the name of the stash. You can use the `git stash list` command to get the name of the stash.

Applying and Drop a Stash

You can apply and drop the stash by using the following command:

```
git stash pop
```

This command applies the stash and drops it from the stash list.

Drop the stash

You can drop the stash by using the following command:

```
git stash drop
```

Applying stash to a specific branch

You can apply the stash to a specific branch by using the following command:

```
git stash apply stash@{0} <branch-name>
```

Clearing the stash

You can clear the stash by using the following command:

```
git stash clear
```

Git Tags

Tags are a way to mark a specific point in your repository. They are useful when you want to remember a specific version of your code or when you want to refer to a specific commit. Tags are like sticky notes that you can attach to your commits.

Creating a tag

You can create a tag using the following command:

```
git tag <tag-name>
```

This command creates a new tag with the specified name. The tag will be attached to the current commit.

Create an annotated tag

You can create an annotated tag using the following command:

```
git tag -a <tag-name> -m "Release 1.0"
```

This command creates an annotated tag with the specified name and message. The tag will be attached to the current commit.

List all tags

You can list all tags using the following command:

```
git tag
```

This command lists all the tags in your repository.

Tagging a specific commit

You can tag a specific commit using the following command:

```
git tag <tag-name> <commit-hash>
```

Push tags to remote repository

You can push tags to a remote repository using the following command:

```
git push origin <tag-name>
```

Delete a tag

You can delete a tag using the following command:

```
git tag -d <tag-name>
```

Delete tag on remote repository

You can delete a tag on a remote repository using the following command:

```
git push origin :<tag-name>
```

Conclusion

In this section, we explored how to use Git's diff, stash, and tags commands. Though not used as frequently as add, commit, or push, they are incredibly helpful in debugging, context switching, and release management. See you next tutorial.

PREVIOUS
Branches in Git

NEXT
Managing History

chaidocs Library Search chaicode

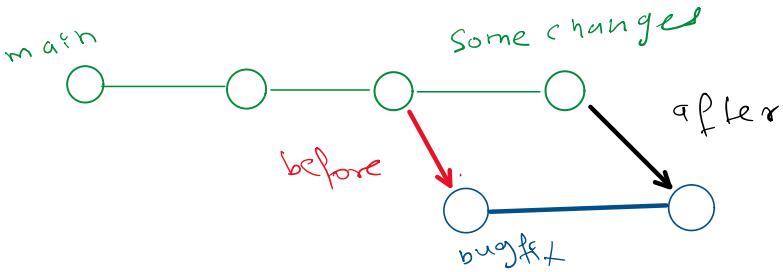
Search docs

MANAGING HISTORY

13 September 2026 23:19

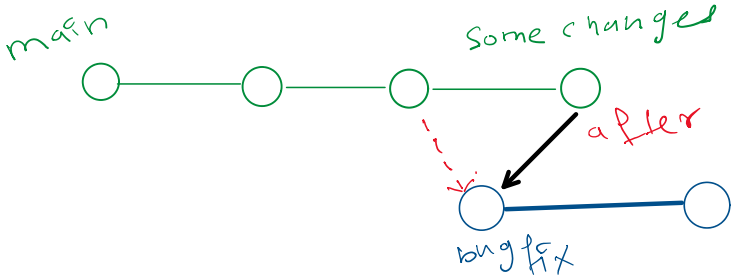
git rebase :

1. should not do on master / main branch
2. we use this because we don't want extra commit



require extra commit to resolve conflict

But what if



Chai aur Git

Managing History

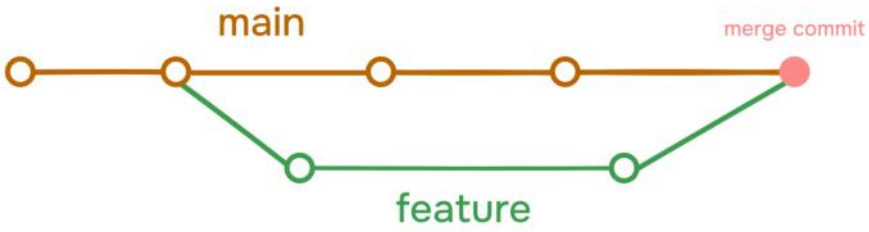
Hitesh Choudhary 5 pages 3 min read Updated Aug 9, 2026

On this page

This guide will help you understand how to manage your Git history effectively.

Merge commits

A merge commit is a commit that combines two or more commits into one. It is created when you merge two or more branches into a single branch. The merge commit contains all the changes from the original branches, and it is used to keep the project history clean and easy to understand.



Rebase in git

Git rebase is a powerful Git feature used to change the base of a branch. It effectively allows you to move a branch to a new starting point, usually a different commit, by "replaying" the commits from the original base onto the new base. This can be useful for keeping a cleaner, linear project history.

Some people like to use rebase over the merge command because it allows you to keep the commit history cleaner and easier to understand. It also allows you to make changes to the code without affecting the original branch.

Here's a flow example of using git rebase with all the commands involved:

Suppose you have a feature branch called feature-branch that you want to rebase onto the main branch.



Ensure you are on the branch you want to rebase

☆ Important

```
git checkout feature-branch
git rebase main
```

This will replay the commits from feature-branch on top of the latest changes in main.

Resolve any conflicts

If there are any conflicts, you will need to resolve them manually. You can use the merge tool in VSCode to resolve the conflicts.

```
git add <resolved-files>
git rebase --continue
```

> Try to avoid --force option when using rebase. It can cause issues with the project history. I have seen many horror stories of people using --force to fix conflicts.

Git reflog

Git reflog is a command that shows you the history of your commits. It allows you to see the changes that you have made to your repository over time. This can be useful for debugging and understanding the history of your project.

View the reflog:

```
git reflog
```

This will show you the history of your commits. You can use the number at the end of each line to access the commit that you want to view.

chaidocs Library Search chaicode Search docs

You can find a specific commit using the following command:

```
git reflog <commit-hash>
```

Recover lost commits or changes

If you accidentally deleted a branch or made changes that are no longer visible in the commit history, you can often recover them using the reflog. First, find the reference to the commit where the branch or changes existed, and then reset your branch to that reference.

```
git reflog <commit-hash>
```

```
git reset --hard <commit-hash>
```



or you can use `HEAD@{n}` to reset to the nth commit before the one you want to reset to.

```
git reflog <commit-hash>
git reset --hard HEAD@{1}
```

Conclusion

In this guide, we've covered important aspects of managing Git history through rebase and reflog. We've learned how rebase can help maintain a cleaner, more linear project history, and how reflog can help recover lost commits or changes.

PREVIOUS
Diff, Stash and Tags

NEXT
About Github

GitHub

14 September 2026 00:26